



AI Driven SQA: Manual & Automation Testing

Assignment 04

Module: 04

**Topic: Introduction to Programming Language
with AI (JavaScript + Node.js)**

Submitted by

Name: Tahsinur Rahman Tanjim

Batch: 21

Phone: 01790576143

Q2. Introduction to Node.js

1. What is Node.js?

Node.js is a free and open-source tool that works on different types of computers. It lets programmers run JavaScript code in places, than a web browser. This tool uses the V8 JavaScript engine from Google Chrome. The V8 engine takes JavaScript code. Turns it into machine code so it can run very quickly.

Node.js is mainly used for developing server-side applications, web servers, APIs, real-time applications, and backend services. It follows an event-driven, nonblocking I/O model, which allows it to handle many requests efficiently without waiting for one operation to finish before processing another.

Node.js also provides npm (Node Package Manager), which is one of the largest software package ecosystems. Developers can use npm to install and manage thousands of ready-made libraries and tools.

Key Features of Node.js:

- Open-source and free to use
- Cross-platform support
- Fast execution using the V8 engine
- Event-driven and non-blocking architecture
- Supports real-time applications
- Large package ecosystem through npm
- Suitable for building scalable web applications and APIs

2. How Does Node.js Work?

Node.js works by using the V8 JavaScript engine, an event-driven architecture, and a non-blocking I/O model. Unlike traditional server-side technologies that create a separate thread for each request, Node.js uses a single main thread with an event loop to efficiently handle multiple requests.

Working Process of Node.js

- A client like a web browser or a mobile app sends a request to the Node.js server.
- The Node.js server gets the request that comes in and figures out what needs to be done.
- Node.js uses an event loop to deal with requests. Of waiting for a slow task to finish Node.js keeps working on other tasks.
- Tasks, like reading files connecting to a database or sending a network request happen in a way that does not stop the app. Node.js does not pause the program while waiting for these tasks.
- Some tasks are handled by Node.js using the system it runs on and a library called libuv. These tasks run in the background so the app can keep going.

Example:

Suppose a user requests information from a database:

Client Request → Node.js Server → Event Loop → Database Operation → Result → Node.js Server → Client Response

While the database is processing the request, Node.js can continue handling other incoming requests instead of waiting.

3. Advantages of Node.js for QA Automation

Node.js provides several advantages for Quality Assurance (QA) automation, especially for web application and API testing. Its large ecosystem, asynchronous architecture, and support for modern automation frameworks make it useful for developing efficient and scalable automated test suites.

1. Easy JavaScript-Based Automation

QA engineers can write automation scripts using JavaScript, a widely used programming language for web development. This makes it easier for teams with JavaScript knowledge to develop and maintain automated tests.

2. Support for Popular Testing Frameworks

Node.js supports many powerful testing and automation tools, such as Playwright, Cypress, WebdriverIO, and Selenium WebDriver. These frameworks can be used for functional, UI, end-to-end, and cross-browser testing.

3. Large NPM Ecosystem

Node.js includes npm (Node Package Manager), which provides access to thousands of packages. QA engineers can use ready-made libraries for assertions, reporting, test data generation, API testing, file handling, and other automation requirements.

4. API Testing Support

Node.js is highly suitable for API automation testing. Tools and libraries can be used to send HTTP requests, validate responses, check status codes, and verify JSON data.

4. Difference between Browser JavaScript and Node.js

JavaScript can run in both web browsers and Node.js, but the environment and available features are different. Browser JavaScript is mainly used to create interactive and dynamic web pages, whereas Node.js allows JavaScript to run outside the browser.

Aspect	Browser JavaScript	Node.js
Execution Environment	Runs inside a web browser such as Chrome, Firefox, Edge, or Safari	Runs outside the browser as a standalone runtime environment
Main Purpose	Used mainly for frontend and web-page interactions	Used mainly for backend applications, APIs, servers, and automation
JavaScript Engine	Uses the JavaScript engine provided by the browser, such as Chrome's V8	Uses Google's V8 JavaScript engine
DOM Access	Can directly access and manipulate the DOM	Does not have a built-in DOM
Browser APIs	Can use APIs such as window, document, and localStorage	These browser-specific APIs are generally unavailable
File System Access	Restricted for security reasons	Can access files using Node.js modules such as fs